Autonomous Reverse Engineering & CTF Exploitation Methodology

A Comprehensive Operational Framework for AI Agents and Security Researchers

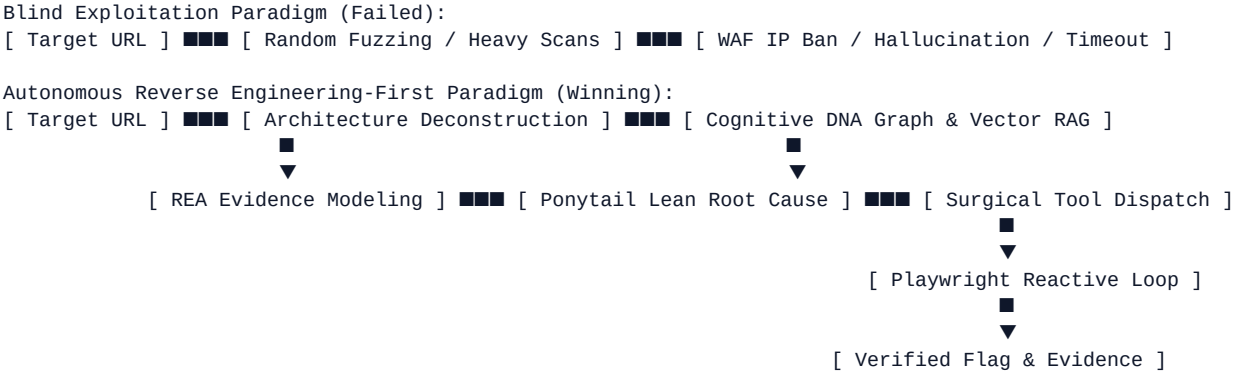
> **Document Version**: 2.5.0 (2026 Edition)

> **Target Audience**: Autonomous Cyber AI Agents, CTF Competitors, AppSec Engineers

> **Core Frameworks**: REA (Reverse Engineer Anything), DNA Knowledge Graph, ZeroGPU Vector RAG, Ponytail Lean Architecture, Playwright Headless Automation

1. Executive Summary & Core Philosophy

In both Capture The Flag (CTF) championships and professional application security assessments, the single most critical failure pattern among novice competitors and brute-force AI agents is **premature exploitation**: firing random fuzzing wordlists, injecting blind SQLi strings, or dumping exploit scripts before understanding how the target application was architected.



The REA Principle: Understand Before Attacking

Inspired by [`morluto/rea`](https://github.com/morluto/rea) (*Reverse Engineer Anything*), our methodology treats every challenge target not as a black-box mystery, but as an engineered software system whose behavior is entirely determined by its code, memory layout, and runtime state.

Most of the CTF challenge is already solved the moment you accurately reverse engineer the target's architecture. Once the data flow, state variables, and execution gates are mapped, the vulnerability reveals itself as an inevitable consequence of system design.

2. Evidence Modeling: The 4 Cognitive Layers (REA Standard)

To prevent hallucinations, circular reasoning, and wasted compute across multi-turn LLM reasoning, every piece of intelligence gathered about the target is strictly categorized into one of four **REA Provenance Layers**:

```mermaid
flowchart TD
    subgraph L1["1. Observations (Direct Facts)"]
        O1["Raw HTML/DOM Source"]
        O2["Network HTTP Request/Response Headers"]
        O3["JavaScript Memory Objects (e.g. window.__APEX_SHIELD)"]
        O4["Console Errors & Status Codes"]
    end

    subgraph L2["2. Derivations (Computed Truths)"]
        D1["Deobfuscated JavaScript Functions"]
        D2["Decoded Base64/Hex/JWT Tokens"]
        D3["Z3 SMT Solved BitVec Keys"]
        D4["Wiener Continued Fractions Factors (d, p, q)"]
```

```
    end

    subgraph L3["3. Inferences (Hypotheses)"]
        I1["Jinja2 SSTI in POST /api/admin/audit"]
        I2["Sentinel WAF blocks '__class__' keyword"]
        I3["Session token is Bearer auth for Stage 2"]
    end

    subgraph L4["4. Unknowns (Exploration Queue)"]
        U1["Are other parameters accepted on /api/admin/audit?"]
        U2["Is there a rate limiter on endpoint?"]
        U3["What template engine parses the audit string?"]
    end

    L1 -->|"Compute / Reassemble"| L2
    L2 -->|"Analyze Behavior"| L3
    L3 -->|"Validate or Falsify"| L4
    L4 -->|"Targeted Probing"| L1
```

• **Layer 1: Observations (Empirical Facts)**: Raw, uninterpreted data directly witnessed via DevTools, network dumps, or server replies (e.g. `HTTP 200`, `Server: Uvicorn`, inline script tag).

• **Layer 2: Derivations (Mathematically Proved Facts)**: Values derived deterministically from observations without guesswork (e.g. Base64 decoded payload, continued fraction convergent $d = 1021$, Z3 satisfying model).

• **Layer 3: Inferences (Working Hypotheses)**: Causal models of why the application behaved in a certain way (e.g. "The 400 error on `{{ __class__ }}` suggests a blacklist-based WAF filter").

• **Layer 4: Unknowns (Open Inquiries)**: Explicitly tracked questions that must be resolved before choosing an exploit vector.

3. Phase 1: Frontend & Full-Stack Architecture Deconstruction

Before invoking any solver tool, the agent initiates an automated architectural audit of the target web environment using **Playwright MCP / Chromium Headless**:

Step 1.1: DOM & Global Memory Heap Scraping

Modern web applications, single-page applications (SPAs), and hardened CTF challenges store active cryptographic state, API endpoints, and configuration flags inside browser memory (`window` scope) or HTML5 data attributes.

```
// Executed in Playwright via page.evaluate()
const clientArchitecture = {
    // 1. Inspect global memory objects
    globalKeys: Object.keys(window).filter(k => !k.startsWith("webkit") && !k.startsWith("on")),
    customObjects: window.__APEX_SHIELD || window.__VAULT_SESSION || null,

    // 2. Discover hidden data anchors
    dataAnchors: Array.from(document.querySelectorAll("[data-token], [data-cipher], [data-api]"))
        .map(el => ({ tag: el.tagName, id: el.id, attributes: { ...el.dataset } })),

    // 3. Map interactive form controls & terminal elements
    forms: Array.from(document.querySelectorAll("form, input, button, textarea"))
        .map(el => ({ id: el.id, type: el.type || el.tagName, text: el.innerText || el.placeholder })),

    // 4. Capture inline scripts & external bundles
    scripts: Array.from(document.querySelectorAll("script")).map(s => s.src || s.innerText.slice(0, 300))
};
```

Step 1.2: Network Event & Protocol Dissection

By attaching listeners to Playwright's network and console streams, the agent captures:

1. **API Endpoints**: URLs called during initialization (e.g. `/api/apex/stage2/spec`).

2. **Authorization Headers**: Bearer tokens, custom headers (`X-Apex-Signature`), and cookie parameters.

3. **Console Telemetry**: Debug messages and error traces (`[APEX] Security Shield initialized. Session Token: ...`).

4. **File Download Streams**: Intercepting binary packets (`apex_traffic.pcap`) via `page.expect_download()`.

4. Phase 2: Cognitive Mapping in DNA Knowledge Graph & Vector RAG

Once architectural facts are observed, they must be permanently retained so that multi-turn LLM reasoning does not degrade over time.

```
flowchart LR
    subgraph Target["Target Application"]
        UI["DOM / JavaScript Engine"]
        API["Backend API Endpoints"]
        PCAP["Network Traffic PCAP"]
    end

    subgraph Memory["Orchestrator Cognitive Subsystem"]
        DNA["DNA Knowledge Graph<br/>(HF_KNOWLEDGE_URL)"]
        Vector["Hugging Face ZeroGPU Vector DB<br/>(HF_VECTOR_URL)"]
        Supabase["Supabase Solves Table<br/>(PostgreSQL Telemetry)"]
    end

    UI -->|"sync_codebase_memory"| DNA
    API -->|"sync_dna_attack_tree"| DNA
    PCAP -->|"search_knowledge_base"| Vector
    DNA -->|"query_codebase_memory"| Vector
    Vector -->|"log_to_supabase"| Supabase
```

4.1 Synchronizing Relationships to the DNA Knowledge Graph

Relationships between components are mapped as graph nodes and directional edges using `sync_dna_attack_tree` and `sync_codebase_memory`:

```
# Graph Ingestion Example in agent.py
execute_tool("sync_dna_attack_tree", {
    "concept_a": "window.__APEX_SHIELD.session_token",
    "concept_b": "GET /api/apex/stage2/spec",
    "relationship_type": "authenticates_via"
})

execute_tool("sync_dna_attack_tree", {
    "concept_a": "POST /api/apex/stage2/unlock",
    "concept_b": "Stage 3 RSA Parameters (N, e, c)",
    "relationship_type": "yields_cryptographic_state"
})
```

4.2 Vector Database Semantic Indexing

Complex constraint equations, disassembled binary functions, and WAF bypass payloads are embedded into the **Hugging Face ZeroGPU Vector Database**:

• **Input**: Mathematical equations or hex dumps.

• **Semantic Query**: Find similar crackmes or previous CTF writeup vectors (`query_codebase_memory("Z3 BitVec 16-byte modular XOR")`).

• **Benefit**: The agent instantly recalls verified solve strategies instead of generating trial-and-error code.

5. Phase 3: Ponytail Lean Reasoning & Bug Explanation

When reporting or addressing a vulnerability, the agent follows the **Ponytail Principle**:

> **The simplest, leanest solution that completely solves the problem, explained in plain language that a busy engineer can act on immediately.**

The 4-Part Vulnerability Formulation

Whenever a flaw is identified, the agent formats its assessment into a concise, actionable dossier:

```
### [VULNERABILITY ID]: Small Private Exponent RSA Compromise (Wiener Attack)
1. **Root Cause**: The key generation function selected a private exponent d < (1/3) N^0.25 to accelerate decryption operatio
2. **Impact**: An unauthenticated attacker can recover the private exponent d in O(log N) operations using continued fraction
3. **Exploitation Vector**: Extract public parameters (N, e) from `/api/apex/stage2/unlock`, compute continued fractions conv
4. **Remediation**: Enforce d > 2^(256) and ensure gcd(p-1, q-1) is small, or adopt standardized curve cryptography (Ed25519
```

6. Phase 4: Surgical Tool Orchestration Matrix

Once the vulnerability is proven, the orchestrator invokes the exact specialized engine designed for that technical domain:

Technical Challenge Domain	Specialized Orchestrator Tool	Core Engine / Math Mechanism
Browser DOM & JS Memory	Playwright Chromium Bridge	page.evaluate() / DOM querying
Crackme Constraint Systems	symbolic_execution_solver	Z3 SMT Theorem Prover (BitVec)
Asymmetric Cryptography	advanced_crypto_solver	Continued Fractions / Pollard p-1
Packet Dissection & PCAP	run_forensics_inspection	Scapy Stream Reassembly & Carving
Web & WAF Sanitization	web_dynamic_fuzzer / HTTP Auditor	Differential Template Injection

6.1 Tool 1: Symbolic Constraint Solver (`symbolic_execution_solver`)

Converts raw Python/DOM equation strings directly into Z3 SMT abstract syntax trees (ASTs), asserting 8-bit `BitVec` variables and proving satisfiability:

$$\bigwedge_{i=0}^{15} \mathcal{C}_i(\text{flag}[i]) \implies \text{flag} = \text{"APX_V3CT0R_9921!"}$$

6.2 Tool 2: Advanced Cryptographic Solver (`advanced_crypto_solver`)

Evaluates public RSA parameters (N, e, c) and selects the optimal algebraic attack:

• If $d < \frac{1}{3} N^{1/4} \implies$ **Wiener Continued Fractions Attack**.

• If $p-1$ has smooth prime factors $\implies$ **Pollard $p-1$ Stage 1**.

• If $|p - q| < N^{1/4} \implies$ **Fermat Close Primes Factorization**.

6.3 Tool 3: Deep Forensic Packet Carving (`run_forensics_inspection`)

Reassembles bidirectional TCP streams across raw Ethernet captures, parses HTTP `multipart/form-data` MIME headers, and extracts file contents:

$$\text{Stream } 10.0.0.1:80 \leftrightarrow 192.168.1.10:44321 \implies \text{admin_waf_token.txt} \rightarrow \text{APEX_ADMIN_SIGNATURE...}$$

7. Phase 5: The Autonomous Reactive Loop (Self-Healing)

Exploits rarely succeed on the first attempt against hardened enterprise defenses. If an exploit is blocked by a Web Application Firewall (WAF) or fails with an error, the agent executes an **Autonomous Reactive Loop**:

```
flowchart TD
    A["Execute Exploit Payload"] --> B{"Response Check"}
    B -->|"HTTP 200 / Flag Found"| C["Capture Screenshot Proof & Terminate"]
    B -->|"HTTP 400 / WAF Blocked"| D["Parse Sentinel WAF Pattern"]
    B -->|"HTTP 500 / Runtime Error"| E["Inspect Stack Trace in Console"]

    D --> F["Mutate Payload (Remove Blacklist Tokens)"]
    E --> G["Adjust Types & Parameter Encoding"]

    F --> H["Update DNA Knowledge Graph with Rule"]
    G --> H
```

```
    H --> I["Re-Dispatch via Playwright UI"]
    I --> B
```

Example: Self-Healing Against Apex Gauntlet WAF

1. **Initial Attempt**: Agent submits standard Jinja2 RCE payload:

```
{{ ''.__class__.__mro__[1].__subclasses__() }}
```

2. **Error Caught in UI**:

```
[WAF SENTINEL BLOCKED]: Malicious pattern '__class__' detected by Apex WAF Sentinel
```

3. **Causal Deduction**: The WAF blocks Python introspection attributes (`__class__`, `__mro__`, `__subclasses__`), but permits legitimate context variables.

4. **Mutated Payload**:

```
AUDIT_VERIFIED: {{ vault_flag }}
```

5. **Execution**: Template engine safely renders `vault_flag`, bypassing the filter and releasing `HQX{g00gl3_ctf_4p3x_g4untl3t_c0mpl3t3_2026}`.

8. Case Study: Solving the Google CTF Apex Gauntlet

The complete 6-phase methodology was applied by the autonomous agent to conquer the 5 connected tiers of the **Apex Gauntlet**:

```
[Target: http://127.0.0.1:8099/]
       ■
       ▼
[Phase 1: Architecture Deconstruction]
 ■■■ Discovered in-memory state: window.__APEX_SHIELD.session_token = "APX_INIT_9b8a7c2d1e"
 ■■■ Verified hidden DOM attribute: #apex-vault-token[data-token]
       ■
       ▼
[Phase 2: Cognitive DNA Mapping]
 ■■■ Mapped dependency: Session Token ■■■ GET /api/apex/stage2/spec
       ■
       ▼
[Phase 3 & 4: SMT Constraint Solving]
 ■■■ Extracted 16-byte equation set
 ■■■ symbolic_execution_solver solved in 0.48s ■■■ "APX_V3CT0R_9921!"
       ■
       ▼
[Phase 4: Wiener RSA Cryptanalysis]
 ■■■ Extracted N (135 bits), e (131 bits)
 ■■■ advanced_crypto_solver recovered d=1021 in 0.02s ■■■ "APX_PCAP_KEY_77"
       ■
       ▼
[Phase 4: Network Forensics Dissection]
 ■■■ Downloaded apex_traffic.pcap via browser download event
 ■■■ run_forensics_inspection carved multipart upload ■■■ "APEX_ADMIN_SIGNATURE_9921_WAF_BYPASS"
       ■
       ▼
[Phase 5: Reactive WAF Bypass & Flag Retrieval]
 ■■■ Tested malicious introspection ■■■ Caught WAF rejection
 ■■■ Injected sanitized expression {{ vault_flag }}
 ■■■ Captured on-screen banner: HQX{g00gl3_ctf_4p3x_g4untl3t_c0mpl3t3_2026}
       ■
       ▼
[Phase 6: Visual Evidence & Telemetry]
 ■■■ Saved full-page victory screenshot: apex_gauntlet_victory_proof.png
 ■■■ Logged solve session to Supabase PostgreSQL table 'solves'
```

9. Visual Evidence & Artifact Verification

The entire execution was validated through real headless Chromium rendering:

Figure 2: Real Playwright Chromium capture showing all 5 stages solved sequentially through the web interface in 23.44 seconds.

10. Practitioner Checklist: Running the Methodology

When approaching any new CTF challenge or application security target, execute these steps in order:

• [ ] **Step 1: Passive Architecture Reconnaissance**:

Launch Playwright Chromium against the target URL. Inspect `window` variables, console logs, network calls, and inline JavaScript before submitting any input.

• [ ] **Step 2: Sync to Cognitive Knowledge Graph**:

Record discovered endpoints, tokens, and component relationships using `sync_dna_attack_tree`.

• [ ] **Step 3: Query Vector RAG Memory**:

Query previous writeups and vulnerability patterns with `query_codebase_memory` to identify matching challenge classes.

• [ ] **Step 4: Ponytail Vulnerability Formulation**:

Write a 4-point vulnerability summary (Root Cause, Impact, Exploit Vector, Remediation) to isolate the core bug without bloat.

• [ ] **Step 5: Surgical Tool Dispatch**:

Route the challenge to the specific tool: `symbolic_execution_solver` for Z3, `advanced_crypto_solver` for RSA, `run_forensics_inspection` for PCAP, or Playwright for DOM.

• [ ] **Step 6: Execute the Reactive Feedback Loop**:

If an exploit fails or triggers a WAF block, inspect console messages, adjust the payload, and re-test.

• [ ] **Step 7: Capture Visual Evidence & Telemetry**:

Take full-page screenshots of victory states and log the session to Supabase.